

Module 1: Remote Procedure Calls & Interface Definition

Remote Procedure Calls (RPC) abstract network communication by allowing a computer program to cause a subroutine to execute in a different address space. Protocol Buffers and gRPC utilize binary serialization over HTTP/2, providing strong typing through an Interface Definition Language (IDL). Marshaling serializes in-memory object graphs into contiguous wire byte arrays, while unmarshaling deserializes byte streams upon reception.

Architectural Takeaways (Module 1): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 2: Logical Time and Vector Clocks

Physical clocks in distributed systems suffer from relativistic drift and quartz oscillator skew. Leslie Lamport introduced logical clocks governed by the happened-before relation ($a \rightarrow b$). Vector clocks extend scalar clocks by maintaining a vector of size N , allowing exact identification of concurrent events and causal dependencies across nodes.

Architectural Takeaways (Module 2): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 3: Distributed Mutual Exclusion

Distributed mutual exclusion coordinates access to shared resources without centralized state. The Ricart-Agrawala algorithm requires $2(N-1)$ messages per critical section entry by broadcasting timestamped request messages. Maekawa's algorithm uses voting sets of size $O(\sqrt{N})$, significantly reducing network message overhead but introducing potential deadlock scenarios.

Architectural Takeaways (Module 3): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 4: Leader Election Algorithms

Leader election identifies a coordinator among identical participating servers. The Bully algorithm assumes synchronous crash-stop failure detectors, electing the node with the highest process ID with $O(N^2)$ message complexity. Ring-based elections pass election tokens along an active unidirectional logical ring until the highest identifier circulates completely.

Architectural Takeaways (Module 4): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 5: The Paxos Consensus Protocol

Paxos achieves consensus over an asynchronous network subject to crash-recovery faults. The protocol separates roles into Proposers, Acceptors, and Learners. Phase 1 (Prepare/Promise) establishes the highest proposal number n , while Phase 2 (Accept/Accepted) commits the value once a majority quorum responds.

Architectural Takeaways (Module 5): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 6: Raft Distributed Consensus

Raft decomposes distributed consensus into understandable subproblems: leader election, log replication, and safety invariants. Servers exist in one of three states: Follower, Candidate, or Leader. Heartbeat timeouts trigger randomized election timers, avoiding split-vote ties and guaranteeing that any committed entry exists in future leader logs.

Architectural Takeaways (Module 6): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 7: Byzantine Fault Tolerance (PBFT)

Practical Byzantine Fault Tolerance operates under arbitrary malicious failure models where nodes can forge or tamper with messages. PBFT requires $3f + 1$ total nodes to tolerate f Byzantine nodes. Three phases—Pre-Prepare, Prepare, and Commit—ensure view change consensus using cryptographic digital signatures.

Architectural Takeaways (Module 7): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 8: Consistency Models in Distributed Systems

Strong consistency guarantees like Linearizability enforce real-time global ordering where every read returns the most recent write. Sequential consistency relaxes real-time constraints while preserving program order on each individual thread. Eventual consistency guarantees that in the absence of new updates, all replicas converge to identical state.

Architectural Takeaways (Module 8): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 9: Amazon Dynamo Architecture

Dynamo provides high availability for shopping cart workloads using decentralized peer-to-peer techniques. Consistent hashing with virtual nodes distributes partitions uniformly across commodity servers. Sloppy quorums (N , R , W) and hinted handoff ensure write availability even during transient network partitions, with vector clocks reconciling conflicting writes.

Architectural Takeaways (Module 9): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 10: Google Bigtable Storage Engine

Bigtable is a distributed, multi-dimensional, sorted map indexed by row key, column key, and timestamp. Tablet servers manage immutable SSTables stored in Google File System (GFS) and maintain in-memory MemTables. Writes append to an on-disk Write-Ahead Log (WAL) before updating MemTable, while background compactions merge SSTables and remove tombstones.

Architectural Takeaways (Module 10): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 11: Google Spanner & The TrueTime API

Spanner is the first globally distributed database providing external consistency and multi-datacenter ACID transactions. TrueTime API exposes bounded clock uncertainty [earliest, latest] using GPS receivers and atomic rubidium oscillators. Two-Phase Commit (2PC) over Paxos groups guarantees serializable snapshot isolation globally without centralized lock managers.

Architectural Takeaways (Module 11): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 12: The CAP Theorem & PACELC Framework

Eric Brewer's CAP theorem demonstrates that distributed systems cannot simultaneously achieve Consistency, Availability, and Partition Tolerance. Daniel Abadi's PACELC framework refines this: if there is a Partition (P), how does the system trade off Availability (A) and Consistency (C); Else (E), how does the system trade off Latency (L) and Consistency (C)?

Architectural Takeaways (Module 12): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 13: Conflict-Free Replicated Data Types (CRDTs)

CRDTs enable concurrent updates on multiple replicas without centralized synchronization while mathematically guaranteeing conflict-free convergence. State-based CRDTs (CvRDT) transmit full state and merge using join-semilattice operations (idempotent, commutative, associative). Operation-based CRDTs (CmRDT) stream commutative operations across a reliable causal broadcast channel.

Architectural Takeaways (Module 13): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 14: Two-Phase Commit & Distributed Transactions

Atomic commit across multiple independent resource managers requires coordination protocols. In Two-Phase Commit (2PC), the coordinator broadcasts Prepare; if all participants vote Commit, Phase 2 issues Global Commit. The fundamental flaw of 2PC is blocking: if the coordinator crashes during Phase 2, participants remain indefinitely locked holding transaction locks.

Architectural Takeaways (Module 14): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 15: Distributed Stream Processing & Fault Tolerance

Stream processing systems process unbounded continuous data streams with sub-second latency. Apache Kafka provides partitioned, immutable commit logs with persistent consumer offsets. Apache Flink achieves exactly-once state semantics via asynchronous barrier snapshotting (Chandy-Lamport variant), checkpointing operator state to durable storage.

Architectural Takeaways (Module 15): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.

Module 16: Service Mesh Architecture & Observability

In microservice topologies, a service mesh decouples networking concerns from business logic. Sidecar proxies (such as Envoy) intercept inbound and outbound L7 traffic, providing mutual TLS (mTLS) authentication, rate limiting, and circuit breaking. Distributed tracing propagates W3C TraceContext headers across RPC boundaries, aggregating spans to visualize distributed latency bottlenecks.

Architectural Takeaways (Module 16): Every design decision in distributed systems represents a trade-off between latency, durability, availability, and algorithmic complexity. Systems engineers must evaluate network partition probability and failure modes prior to selecting consensus quorums or replication factors.